Policy-Aware SQL Auditor: Detailed Project Report

Oct 21, 2026 · @Hitesh

1. Summary

This is a local-first text-to-SQL system that checks every LLM-written query against versioned business rules before it runs, so a query that executes but answers the wrong question is caught instead of returned.

The problem. A generated query can be valid SQL and still violate a company definition. If the table has `is_active` but the company defines an active customer as one with a completed order in the last 90 days, a query on `is_active` returns plausible, wrong numbers. Basic text-to-SQL tools do not notice this.

The approach. Business definitions are stored as OKF concept files. The system retrieves the relevant ones, the LLM writes SQL, deterministic code checks safety, a second deterministic check verifies the rule's required filters appear in the parsed SQL, and only then does a read-only executor run it. Every answer carries the SQL, the rule IDs used and the check results.

The hypothesis to test. Rule-aware retrieval plus a deterministic rule audit produces fewer rule-violating and unsafe queries than basic text-to-SQL. This is a hypothesis, not a result: the project is at design stage and no implementation or evaluation numbers exist yet.

Scope. Student MVP on synthetic data, SQLite, free and local tools, one API endpoint.

2. Related work and what is different

The general idea is not new: safety-checked, multi-agent text-to-SQL on SQLite already exists, and Datum is the closest match. The part nobody covers is auditing generated SQL against versioned business rules. Overlap percentages are my own rough estimates, not measured.

Project	Purpose	Est. overlap	Difference from this project
Datum	Grounds an LLM on a governed semantic layer; Planner, SQL Author, Critic, Answerer with repair loop	~70%	Its sqlglot critic blocks unknown columns, PII, uncertified tables and non-read-only SQL (safety and grounding). It does not verify that the SQL applies a business rule's required filters.
sql-analyst-agent	Six agents plan, write, validate, execute and explain SQL with self-correction	~50%	Same safety layers (sqlglot, read-only SQLite, row cap, timeout). No business-rule layer or rule audit.
Wren AI	Open-source generative BI with a version-controlled semantic layer (MDL)	~45%	Definitions guide generation up front; there is no separate post-generation pass/fail audit. A far larger platform.
Vanna	RAG over DDL, documentation and example SQL	~30%	Retrieval only; no deterministic rule check or audit trail. Main repo archived March 2026.
Dataherald	Enterprise NL-to-SQL engine with API, admin console and Slackbot	~30%	Business context for generation; no rule-compliance audit.
sql-guardrail, ProxQL, NL2BI	Safety validation of LLM-written SQL	~25% (guardrail module only)	Stop at safety; none check business-rule correctness.

The defensible difference is five things taken together:

1. A rule-compliance audit: required filters and joins from each rule are checked on the parsed SQL, with a pass or fail and a reason.
2. Business rules stored in OKF, an open standard, instead of a custom format.
3. Rule IDs and versions cited in every answer, plus an audit record.

4. An optional Jev check that returns a thresholdable probability instead of free-text judgment.
5. An evaluation built around queries that run fine but break a rule.

In interviews and the README, name Datum as related work and explain the gap. Study it, but write the code independently.

3. Scope

The MVP is one synthetic business domain on SQLite, answered through a single API endpoint, with the rule audit as its centerpiece.

Goals

- Accept natural-language questions about a known relational database.
- Retrieve relevant schema details and business rules, and ground SQL generation in them.
- Validate SQL before execution: single read-only SELECT, allowlisted tables and columns, row and time limits.
- Check that the SQL applies each retrieved rule's required filters.
- Return the answer with the executed SQL, rule IDs and versions, and check results; write an audit record.
- Compare against two baselines on a fixed, hand-verified test set.
- Run locally with free tools; no paid cloud service is a hard dependency.

Non-goals for the first version

- Other SQL dialects or database engines (PostgreSQL is a later option).
- Discovering rules automatically from arbitrary documents.
- Guaranteeing that an LLM-generated query is correct.
- Replacing database permissions, governance or human review.
- INSERT, UPDATE, DELETE, DDL or stored procedures.
- Multi-turn conversation memory and a polished front end.
- Production use with real or sensitive data.

MVP is done when it runs locally from the README, uses a documented synthetic database, rejects destructive and multi-statement SQL, catches several deliberately built rule violations, cites rule IDs, has tests for success, failure and adversarial cases, and reports measured baseline results with its limitations.

5. Tech stack and how each piece is used

Python end to end, with every component free and local. OKF is the core knowledge format; Jev is an optional component behind an interface, adopted only if an ablation shows it helps.

Layer	Technology	How it is used here
Language	Python 3.11+	All modules; typed inputs and outputs per module
API	FastAPI + Pydantic	`POST /ask` and `GET /health`; Pydantic models define request, response and rule schemas
Database	SQLite (read-only via `mode=ro`)	Synthetic `departments`, `budgets`, `expenses`, `customers`, `orders` data; PostgreSQL is a later option
Knowledge format	OKF bundle (Google Cloud open spec)	Each business rule, metric and table description is one markdown concept file with YAML front matter; the file path is the concept's ID
OKF tooling	`okflib` Python library	Load and validate bundles; its MCP server can expose the bundle to Claude as an extra demo
Retrieval	Metadata filters + BM25 lexical search	Pick rules and schema entries by concept type, tags and keywords; embeddings are optional, not required
SQL parsing	SQLGlot	Single-statement check, SELECT-only check, table and column allowlists, and extraction of WHERE and JOIN conditions for the rule audit
LLM (SQL generation)	Local model via Ollama, or a mock model for development	Only proposes SQL; never executes it. Benchmark one stronger model separately, since small local models are weak at text-to-SQL

Layer	Technology	How it is used here
Router	Rule-based first; Jev optional	Classify the question as metric, policy, ambiguous or unsupported
Semantic review signal	Jev optional	Yes/no probability that the SQL applies a named rule; recorded as a signal, never an approval
Tests	pytest	Unit, integration, safety (attack queries) and evaluation suites
Audit trail	JSONL file or SQLite table	One record per request: rule IDs and versions, candidate and executed SQL, check outcomes, row count

OKF: how we use it. OKF is Google Cloud's open format (June 2026) for knowledge as a directory of markdown concept files with front matter; concepts can be tables, datasets, metrics and playbooks. We keep rules in `knowledge/` as an OKF bundle, load them with `okflib`, and cite concept paths and versions in answers. Still to verify in the spec: whether custom front-matter fields such as `required_filters` are allowed. If not, the machine-readable rule goes in a fenced block in the file body.

Jev: how we would use it. Jev (TypeSafe AI, launched September 2026, early-access API) is a decision model. It does not write text; it answers typed questions with a choice, a score or a yes/no probability plus confidence, at roughly 70 to 500 ms. It fits two narrow roles here: routing the question, and a secondary "does this SQL apply rule X?" check. It cannot write SQL and cannot override a failed deterministic check. Because the API is early-access, both uses sit behind a small interface with a rule-based fallback, and we keep Jev only if the evaluation shows a measurable gain. Confirm access, pricing and terms before building on it.

Cost. No paid service is a hard dependency. If the chosen local model is too slow or weak, use a mock for pipeline development and a hosted model only for the final benchmark run.

6. Knowledge layer: OKF rules and required filters

Each business rule is one OKF concept file, and the rule's machine-checkable part (its required filters) is what the semantic audit verifies against the parsed SQL.

Bundle layout

```
knowledge/
  index.md
  metrics/
    departmental_spend.md
    active_customer.md
  policies/
    expense_reimbursement.md
    fiscal_year.md
  tables/
    departments.md
    budgets.md
    expenses.md
    customers.md
    orders.md
```

Example rule file (illustrative; adjust to the OKF spec after reading it)

```
---
type: Business Metric
title: Departmental spend
description: Approved expenses per department within the fiscal year.
tags: finance, spend
timestamp: 2026-10-21T00:00:00Z
---
# Definition
Spend counts only approved expenses dated inside the current fiscal year.

# Tables
Uses [expenses](/tables/expenses.md) and [budgets](/tables/budgets.md).

# Required filters
- expenses.status = 'approved'
- expenses.expense_date within the fiscal year
  (see [fiscal year](/policies/fiscal_year.md))
```

Design points

1. Stable identity: the file path is the rule ID; the front matter timestamp or a version line tracks changes, and answers cite both.
2. Explicit mappings: every rule names the tables, columns and fiscal-year or window logic it depends on.
3. Machine-readable section: the "Required filters" list is parsed by the loader into typed objects (column, operator, value or range type). A new rule is a new file, not new code.
4. Normalisation: the checker treats `status = 'approved'` and `status IN ('approved')` as the same condition, and handles a few common equivalent forms.

5. Retrieved text is data, not instructions: rule bodies are never executed or obeyed as prompts, which blocks prompt injection through knowledge files.
6. Loader tests: every file must parse, every referenced table and column must exist in the live schema, and every link must resolve.

7. Guardrails, semantic audit and threat model

Safety never depends on the LLM: deterministic checks run first and a failure ends the request, whatever any model says.

Deterministic guardrails (SQLGlot plus database controls)

- Parse the SQL; allow exactly one read-only SELECT (CTEs included, DML hidden inside them rejected).
- Reject multiple statements, DDL, DML, administrative commands and disallowed functions.
- Enforce table and column allowlists; reject identifiers not in the live schema.
- Apply a row cap and a query timeout; open SQLite with `mode=ro` so the engine refuses writes even if a check is bypassed.
- Fail closed: if the parser cannot classify the statement, do not run it.

A keyword denylist alone is not enough, because comments, nested queries and dialect syntax defeat it. Parsing plus a read-only connection is the minimum.

Semantic audit (the project's core)

1. For each retrieved rule, read its required filters.
2. Parse the candidate SQL with SQLGlot and collect WHERE and JOIN conditions.
3. Normalise equivalent forms, then check each required filter is present.
4. Return pass, or fail with a reason such as "missing required filter: expenses.status = 'approved'".
5. On fail, send the reason back to the generator for one controlled revision, ask the user to clarify, or refuse.
6. Optionally record a Jev or LLM review as a secondary signal. It can add a warning but can never approve a query that the guardrails or rule audit rejected.

Worked example: the rule requires approved status and a fiscal-year date range. A query with only the date range returns inflated spend. The audit flags the missing status filter instead of returning the number.

Threat model

Threat	Mitigation
Destructive or multi-statement SQL	Parsed SELECT-only check plus read-only connection
Unauthorized table or column access	Allowlists and least-privilege credentials
Expensive query	Row cap, timeout, synthetic dataset
Prompt injection in knowledge files or the question	Treat retrieved text as data; keep policy content separate from system instructions
Invented tables or columns	Check identifiers against the live schema
Wrong or outdated rule	Versioned rules, explicit mappings, loader tests
Misleading explanation	Ground the answer in executed rows and cited rule IDs
Secrets or sensitive values in logs	Redact; define retention before any real data
Model unavailable	Return a controlled error; never skip validation to keep running

Known limits. The audit only catches rules that can be written as required filters, joins or columns. A read-only query can still expose data to a user who is allowed too much. Rules can be incomplete or stale. Production use would need authentication, authorization, monitoring and a security review.

8. Evaluation plan

The project is only credible if the same fixed test set is run against baselines and the full system, and the real numbers are reported, including failures.

Test set. 50 to 100 synthetic questions, each with a hand-verified gold SQL or expected result. Cover:

- Straightforward filters, aggregations and joins.
- Metrics with explicit definitions and fiscal-year or rolling-window logic.
- Queries that execute but violate a business rule (the project's target gap).
- Ambiguous requests that should trigger clarification.
- Unknown tables or columns, and prohibited operations.
- Prompt-injection attempts in a question or a knowledge file.

- Empty results and database errors.

Document ambiguous cases instead of treating one reading as the only correct one. Freeze and version the set before running experiments.

Configurations compared

ID	Configuration
A	Basic text-to-SQL: question and schema only
B	A plus retrieved business rules, no audit
C	B plus deterministic guardrails and the rule audit (the proposed system)
D	C with Jev as router and review signal (run only if Jev is integrated)

C versus B isolates the value of the audit; D versus C isolates the value of Jev. A separate ablation compares rule-based routing with Jev routing.

Metrics

Metric	What it measures
Execution accuracy	Result matches the expected result on the test database
Business-rule compliance	Query applies the required definitions and filters
Unsafe-query rejection rate	Share of disallowed test queries rejected
False rejection rate	Share of valid supported queries wrongly rejected
Evidence accuracy	Cited rule IDs actually support the explanation
Clarification quality	Ambiguous questions trigger a useful clarification
Latency and resource use	End-to-end time; model tokens or memory where measurable

Exact SQL match is recorded but not used alone, since different SQL can be equally correct. Report the number of test cases, per-category error breakdowns and the weak results. Do not state target scores as achievements before they are measured.

9. Roadmap, repository, risks and sources

Build in the order that proves the core first: data and guardrails, then retrieval, then the rule audit, then evaluation. Cut Jev, the multi-agent split and the UI first if time runs short.

Roadmap, with an exit check for each phase

1. **Data and tests.** Synthetic SQLite database with trap rows (pending expenses, boundary dates), 20 gold questions. Exit: schema, rules and test cases are explicit and reproducible.
2. **Guardrails and executor.** SQLGlot checks, read-only executor, 15+ attack queries. Exit: approved queries run, disallowed ones are rejected, tests pass.
3. **Baseline and OKF retrieval.** Basic text-to-SQL baseline, OKF bundle, loader, BM25 retrieval, rule IDs returned with each answer. Exit: questions retrieve the intended rules.
4. **Rule audit and routing.** Required-filter checker, rule-based router, controlled revise-or-reject. Exit: known rule violations are caught in a regression set.
5. **Evaluation.** Run configurations A to C on the frozen test set; add D if Jev is integrated; categorise errors. Exit: a written report of actual results and limitations.
6. **Polish.** README, architecture and threat-model docs, short demo of a valid query, a caught rule violation and a rejected unsafe query. Exit: a reviewer can run it locally.

Repository layout

```
enterprise-sql-auditor/
  README.md
  pyproject.toml
  knowledge/            # OKF bundle: metrics, policies, tables
  data/seed_synthetic_data.py
  src/sql_auditor/
    api.py  config.py  schema_inspector.py
    knowledge_loader.py  retrieval.py  router.py
    sql_generator.py  sql_guardrails.py
    rule_auditor.py  executor.py
    answer_builder.py  audit_log.py
  tests/ unit/ integration/ safety/ evaluation/
  docs/ architecture.md  threat_model.md  evaluation.md
```

Main risks

Risk	Mitigation
Too much scope	SQLite, one domain, one endpoint, 8 to 15 rules; cut Jev and the UI first

Risk	Mitigation
Closest project (Datum) makes this look like a copy	Focus on the rule-compliance audit and OKF, cite Datum as related work, write the code independently
Weak local model skews results	Test a model early; use a stronger hosted model only for the benchmark run
Jev is early-access and may change	Keep it behind an interface with a rule-based fallback
OKF spec does not allow custom fields	Put machine-readable rules in a fenced block in the file body
Weak evaluation	Freeze a versioned test set with gold results and adversarial cases

Before listing it on a resume

- [ ] Code runs locally from the README and the repo is public.
- [ ] Tests pass, including unsafe-query and rule-violation cases.
- [ ] Baseline comparison done, with real numbers.
- [ ] Every bullet on the resume can be backed by something in the repo.
- [ ] You can explain each design choice, including what did not work.

Sources

- Google Cloud: Open Knowledge Format
- okflib on PyPI
- Jev launch coverage (VKTR)
- Jev primitives and latency (IT-Connect)
- Datum
- sql-analyst-agent
- WrenAI
- Wren AI vs Vanna AI comparison
- Dataherald README
- sql-guardrail, ProxQL, NL2BI